LAPORAN PRAKTIKUM 7 SISTEM INFORMASI GEOGRAFIS

INTEGRASI FASTAPI DAN POSTGIS: MEMBANGUN REST API SPASIAL

Disusun Oleh:
Anselmus Herpin Hasugian (123140020)

Dosen Pengampu:

Muhammad Habib Algifari, S.Kom., M.T.I.

Alya Khairunnisa Rizkita, S.Kom., M.Kom.

PROGRAM STUDI TEKNIK INFORMATIKA

INSTITUT TEKNOLOGI SUMATERA

2026

BAB 1.

PENDAHULUAN

1.1. Latar Belakang

Sistem Informasi Geografis (SIG) modern menuntut arsitektur yang decoupled, di mana basis data spasial (backend) dapat berinteraksi secara mulus dengan antarmuka pengguna (frontend). Untuk menjembatani komunikasi ini, diperlukan sebuah Application Programming Interface (API). Praktikum ini difokuskan pada implementasi REST API menggunakan framework FastAPI (Python) untuk mengekspos data spasial yang tersimpan di dalam PostGIS. Penggunaan FastAPI dipilih karena kecepatannya yang asinkronus (berkat asyncpg), validasi data bawaan via Pydantic, dan kemampuannya men-generate dokumentasi interaktif secara otomatis.

1.2. Tujuan

1. Memahami konsep REST API dan perannya dalam arsitektur WebGIS.
2. Membangun *project* FastAPI beserta konfigurasi *connection pool* ke basis data PostgreSQL/PostGIS.
3. Mengimplementasikan operasi CRUD (Create, Read) melalui *endpoint* GET dan POST.
4. Menghasilkan format luaran berstandar peta web, yaitu GeoJSON.
5. Mengeksekusi *query* spasial langsung dari API (seperti pencarian *nearby* menggunakan ST_DWithin).

BAB 2.

LATIHAN

PERSIAPAN LINGKUNGAN KERJA (SETUP & DEPENDENCIES)

Tahapan awal pengembangan dimulai dengan mengisolasi lingkungan kerja Python menggunakan virtual environment (.venv). Isolasi ini merupakan best practice dalam rekayasa perangkat lunak untuk memastikan bahwa pustaka (library) yang digunakan pada proyek ini tidak mengalami konflik (bentrok versi) dengan aplikasi lain di dalam sistem operasi lokal.2.1. Query Jarak (Fasilitas Publik)

2.1. Inisialisasi Virtual Environment

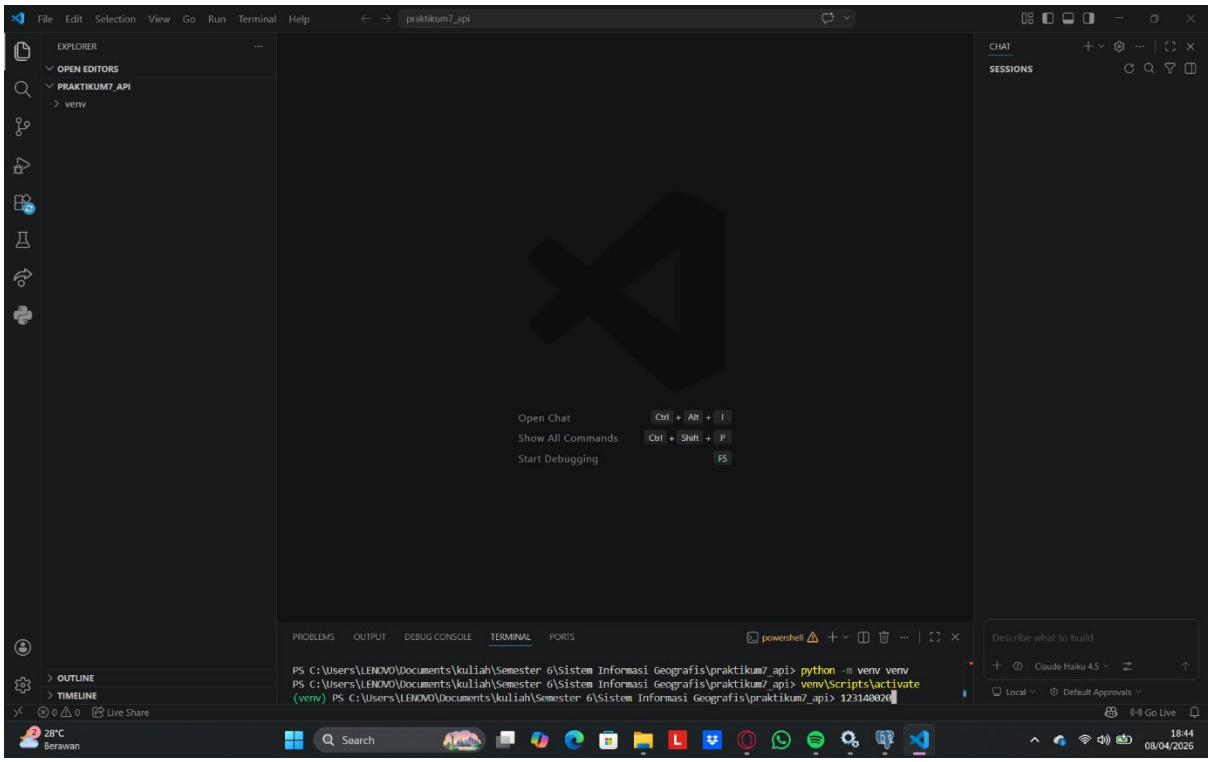
Sebuah virtual environment (.venv) dibangun untuk mencegah konflik dependensi antar project.

Kode:

```
python -m venv .venv

# Aktivasi di Windows

.venv\Scripts\activate
```



models.py > FasilitasCreate

```python
from pydantic import BaseModel

class FasilitasBase(BaseModel):
    nama: str
    longitude: float
    latitude: float

class FasilitasCreate(FasilitasBase):
    pass
```

```
Using cached pydantic-2.12.5-py3-none-any.whl (463 kB)
Using cached pydantic_core-2.41.5-cp313-cp313-win_amd64.whl (2.0 MB)
Using cached annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading click-8.3.2-py3-none-any.whl (108 kB)
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Downloading starlette-1.0.0-py3-none-any.whl (72 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Installing collected packages: typing-extensions, python-dotenv, idna, h11, colorama, asyncpg, annotated-types, annotated-doc, typing-inspection, pydantic-core, click, anyio, uvi
corn, starlette, pydantic, fastapi
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.13.0 asyncpg-0.31.0 click-8.3.2 colorama-0.4.6 fastapi-0.135.3 h11-0.16.0 idna-3.11 pydantic-2.12.5 pydan
tic-core-2.41.5 python-dotenv-1.2.2 starlette-1.0.0 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.44.0

[notice] A new release of pip is available: 25.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\LENOVO\Documents\kuliah\Semester 6\Sistem Informasi Geografis\praktikum7_api>
```

database.py > create_pool

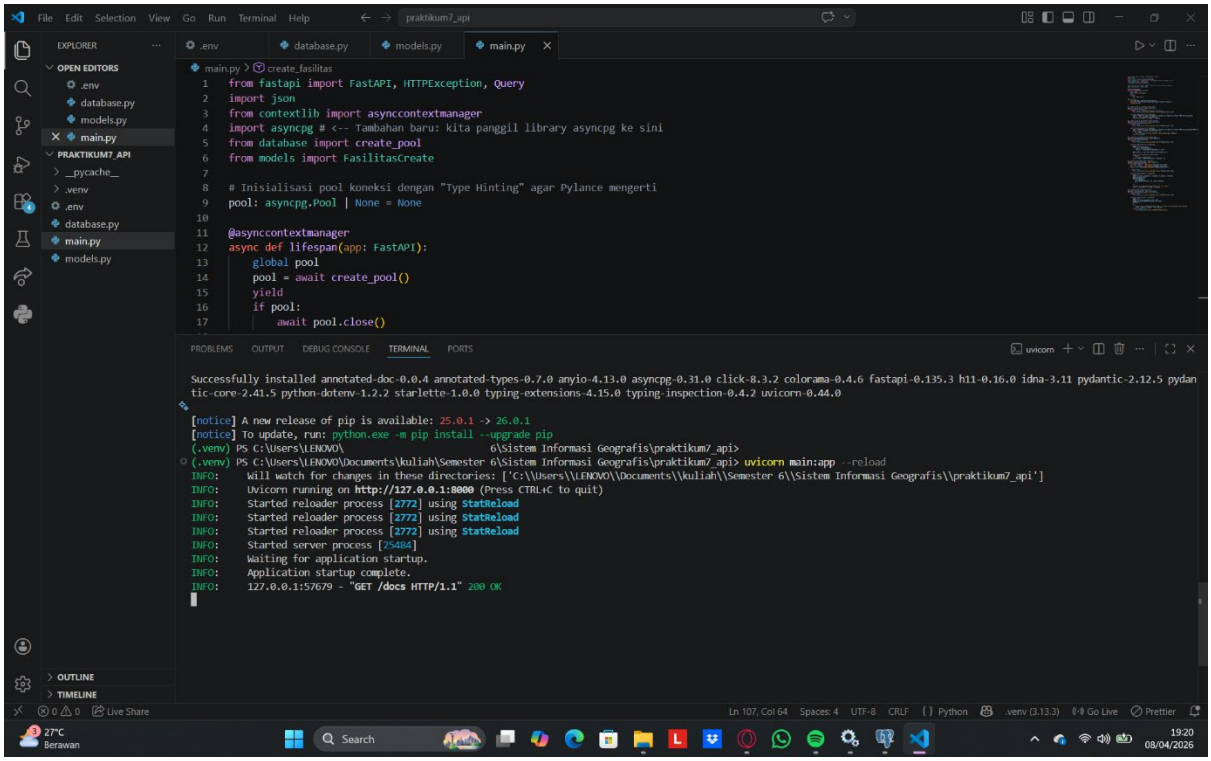
```python
import asyncpg
import os
from dotenv import load_dotenv

load_dotenv()

async def create_pool():
    return await asyncpg.create_pool(
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME"),
        host=os.getenv("DB_HOST"),
        port=os.getenv("DB_PORT")
    )
```


```
Using cached pydantic-2.12.5-py3-none-any.whl (463 kB)
Using cached pydantic_core-2.41.5-cp313-cp313-win_amd64.whl (2.0 MB)
Using cached annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading click-8.3.2-py3-none-any.whl (108 kB)
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Downloading starlette-1.0.0-py3-none-any.whl (72 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Installing collected packages: typing-extensions, python-dotenv, idna, h11, colorama, asyncpg, annotated-types, annotated-doc, typing-inspection, pydantic-core, click, anyio, uvi
corn, starlette, pydantic, fastapi
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.13.0 asyncpg-0.31.0 click-8.3.2 colorama-0.4.6 fastapi-0.135.3 h11-0.16.0 idna-3.11 pydantic-2.12.5 pydan
tic-core-2.41.5 python-dotenv-1.2.2 starlette-1.0.0 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.44.0

[notice] A new release of pip is available: 25.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\LENOVO\Documents\kuliah\Semester 6\Sistem Informasi Geografis\praktikum7_api>
```



Gambar 2.1 Setup

2.2. Instalasi Dependensi Inti

Instalasi pustaka yang dibutuhkan dilakukan menggunakan pip:

1. fastapi & uvicorn: Framework utama dan server ASGI.
2. asyncpg: Driver PostgreSQL asinkronus tercepat untuk Python.
3. pydantic: Validasi skema tipe data.
4. python-dotenv: Pengelola variabel lingkungan dari file .env

Kode:
pip install fastapi uvicorn asyncpg python-dotenv pydantic

2.3. Konfigurasi Koneksi Basis Data

Kredensial PostgreSQL diisolasi di dalam file .env dan dihubungkan melalui Connection Pool di database.py. Konsep connection pooling diterapkan agar aplikasi tidak membuat koneksi baru setiap kali ada request masuk, melainkan menggunakan ulang koneksi yang sudah terbuka, sehingga performa API meningkat drastis saat diakses secara konkuren.

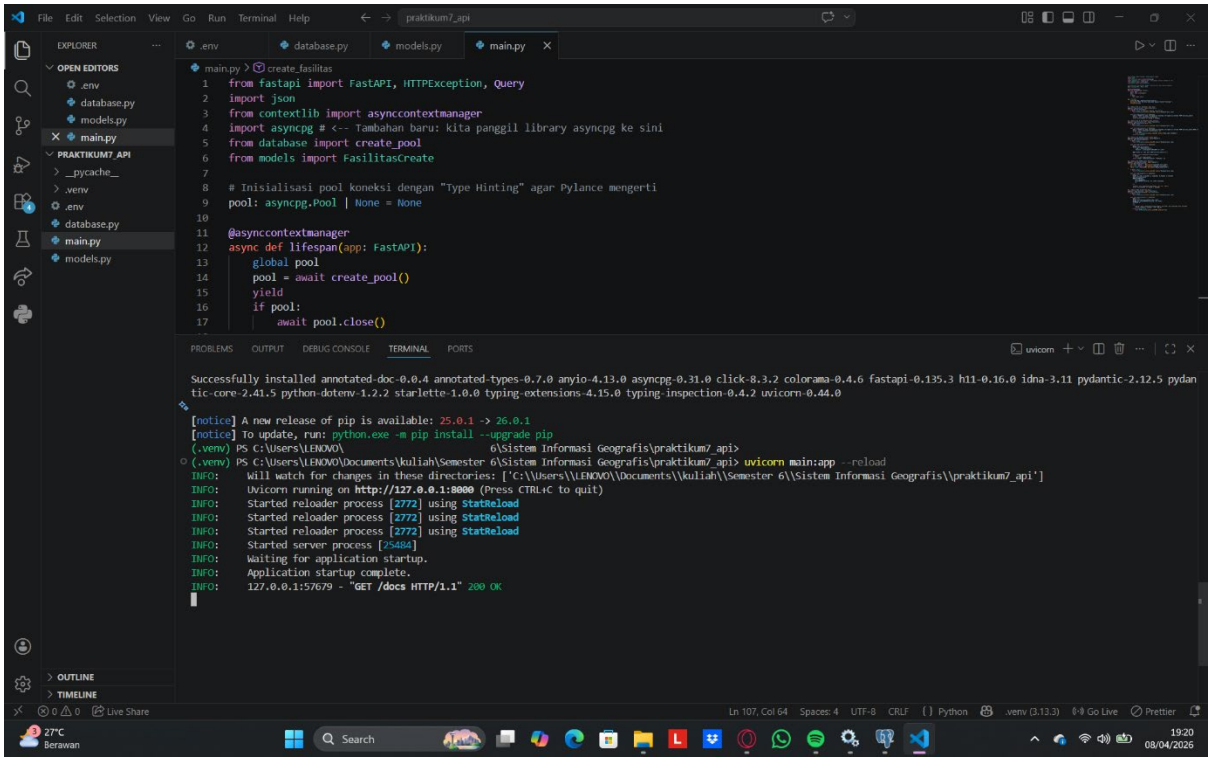
```
EXPLAIN ANALYZE

SELECT w1.nama, w2.nama

FROM wilayah w1, wilayah w2

WHERE ST_Intersects(w1.geom, w2.geom)

 AND w1.id != w2.id;
```

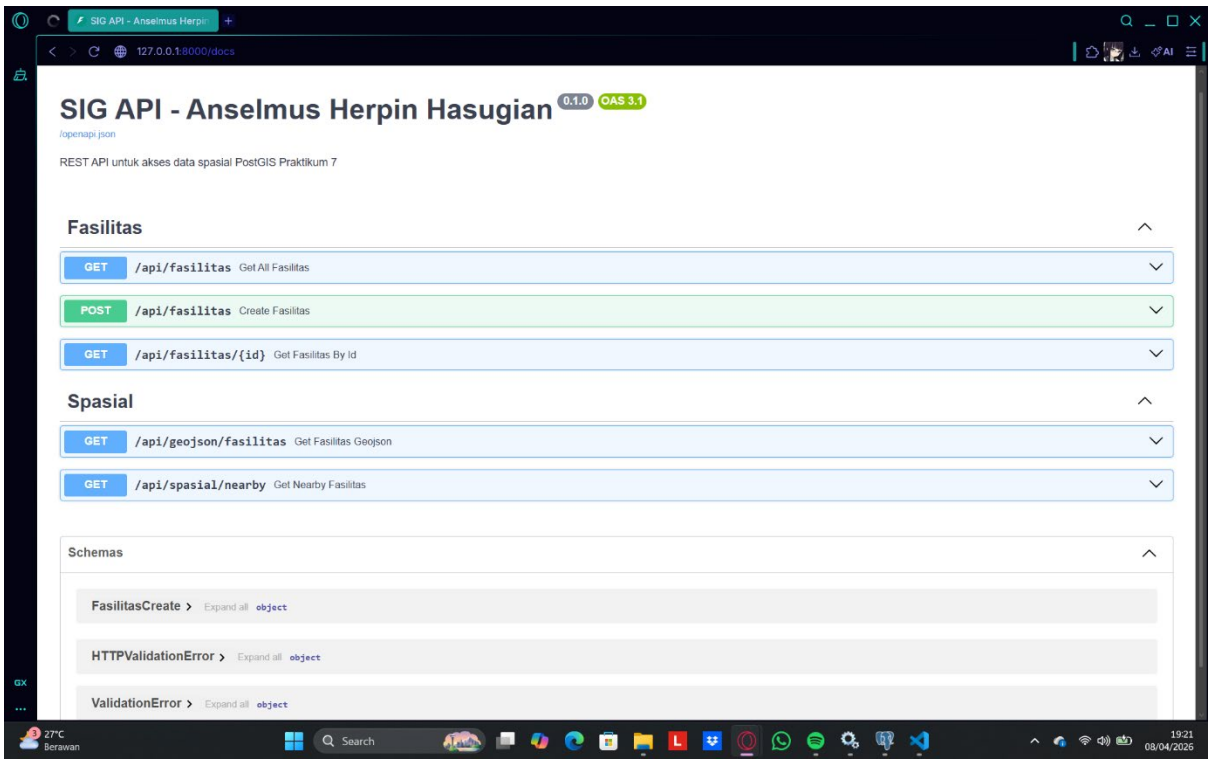


Gambar2.3. Tampilan sukses dari perintah unicorn main:app --reload di terminal

BAB 3.

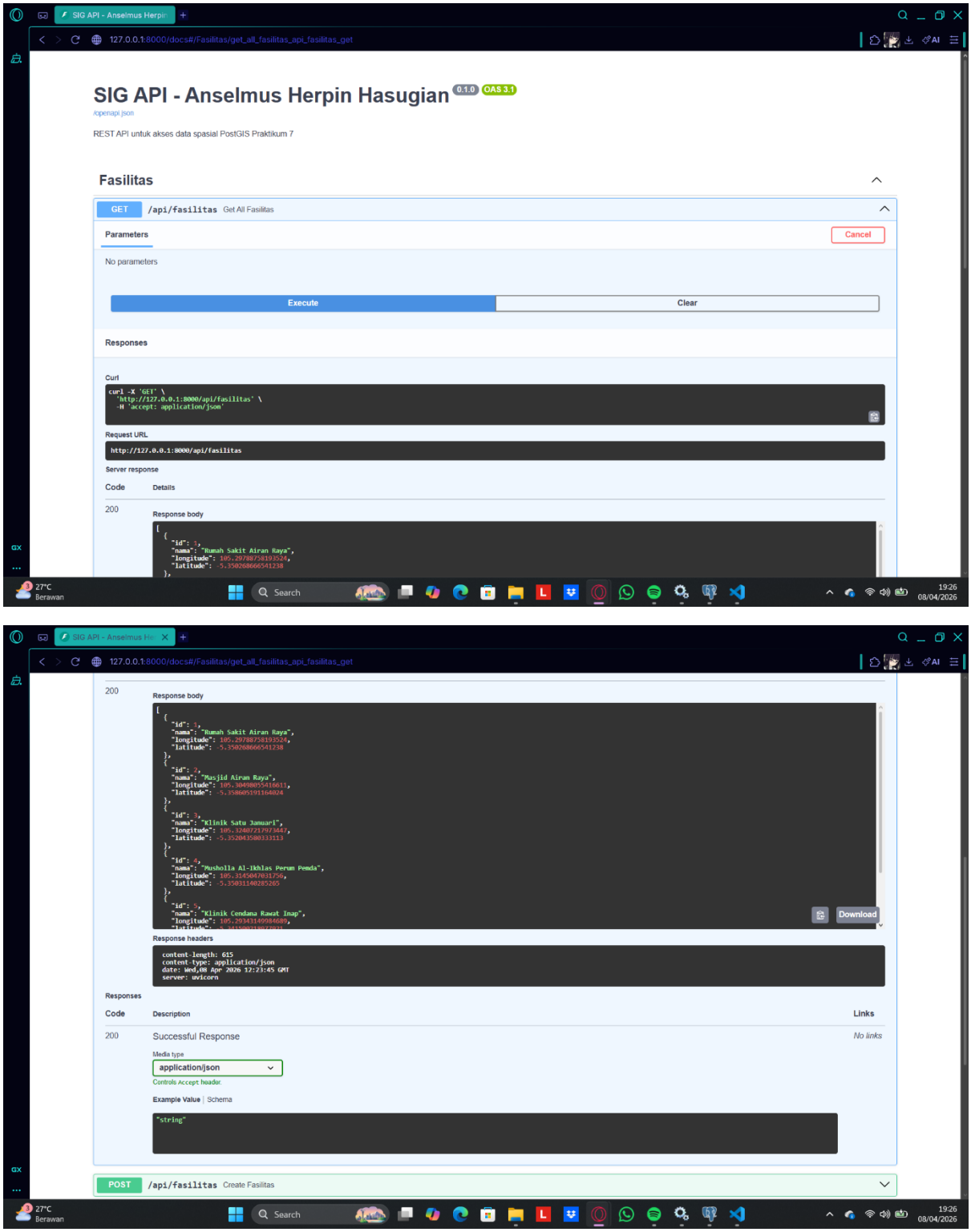
IMPLEMENTASI ENDPOINT DAN ANALISIS SPASIAL

Bagian ini memaparkan realisasi 5 (lima) endpoint wajib beserta dokumentasi pengujiannya menggunakan Swagger UI (/docs).



3.1. Endpoint 1: GET /api/fasilitas (Retrieve All Data)

Antarmuka pertama dibangun untuk mengambil seluruh entitas data spasial fasilitas publik dari tabel PostGIS. Karena data geometri mentah (binary) tidak dapat dibaca langsung oleh aplikasi client, query SQL yang disisipkan dirancang untuk mengekstrak titik koordinat menggunakan fungsi ST_X(geom) sebagai nilai longitude dan ST_Y(geom) sebagai nilai latitude.

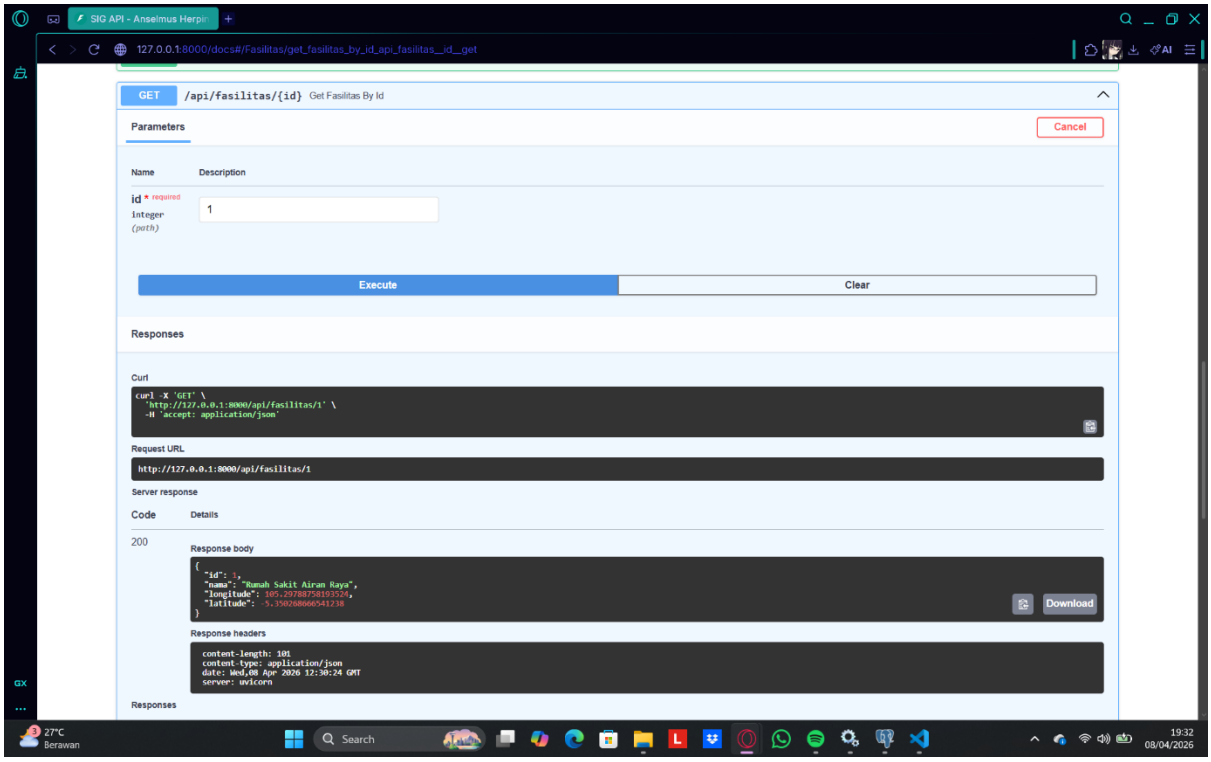


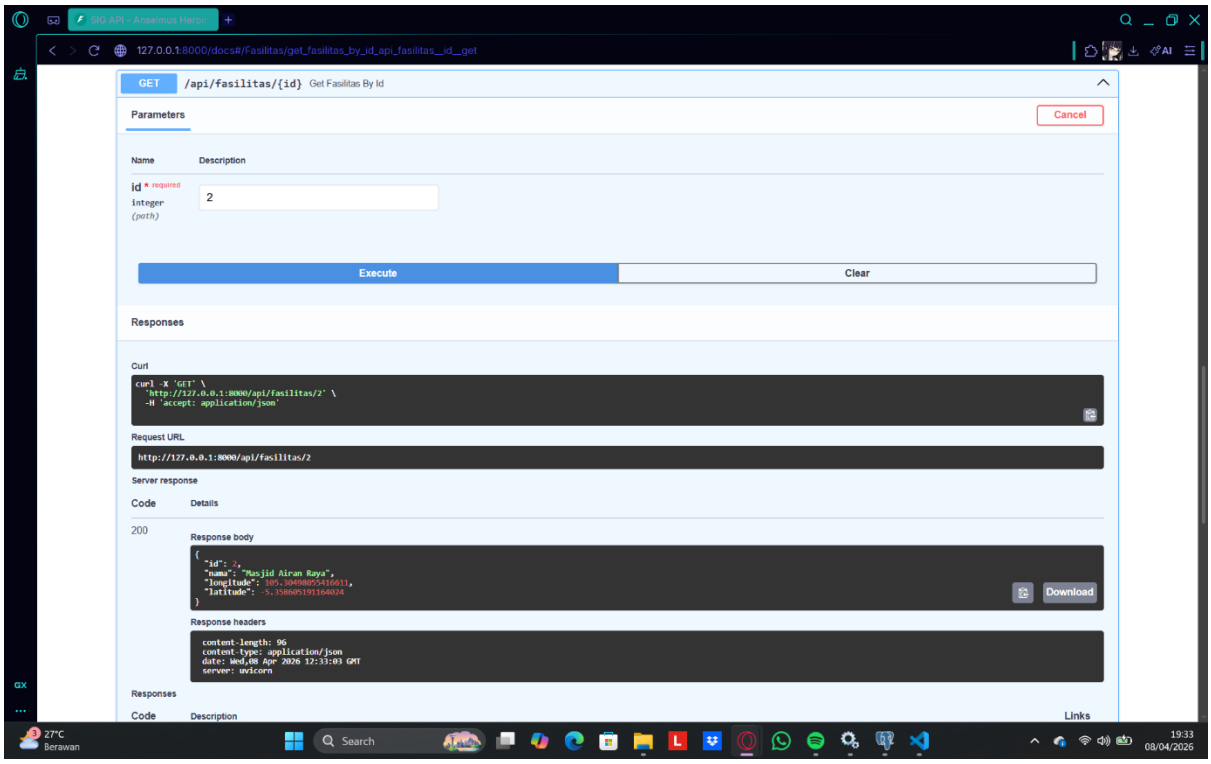
Gambar 3.1. Pengujian Endpoint GET All Fasilitas

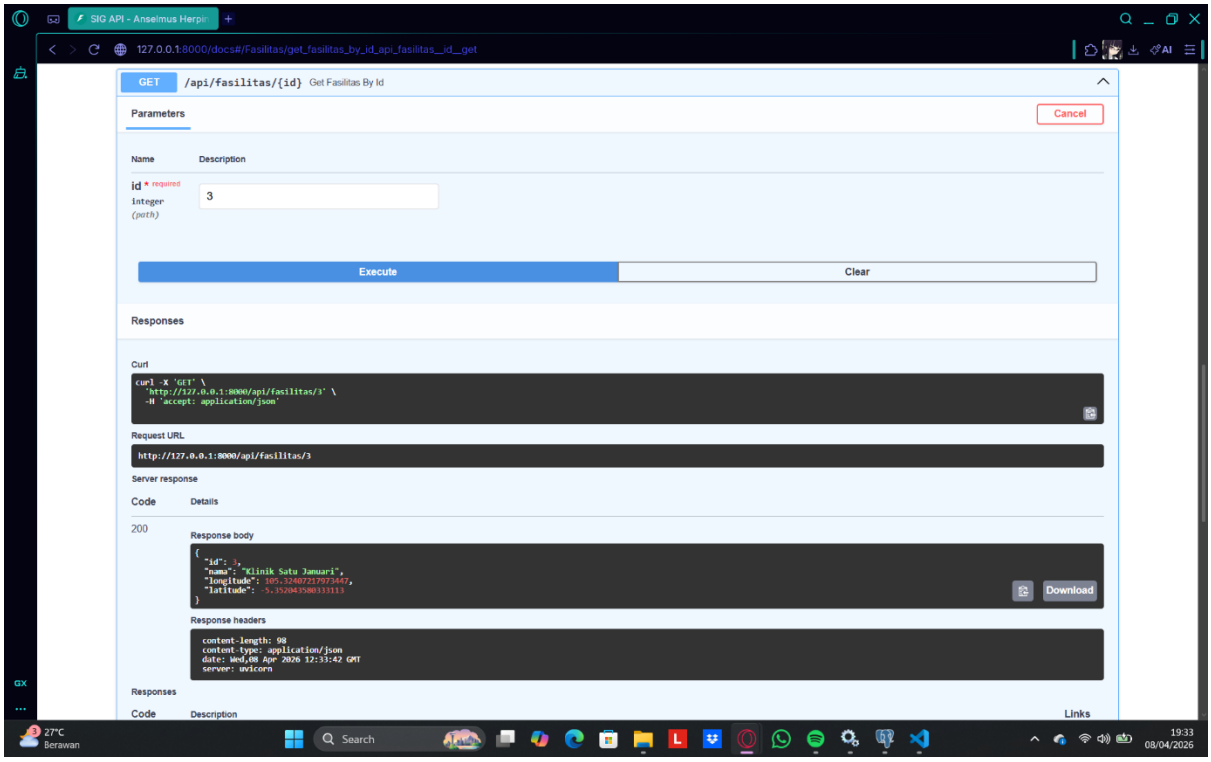
Berdasarkan hasil eksekusi pada Gambar 3.1, server merespons dengan kode HTTP 200 (OK), menandakan komunikasi berhasil tanpa hambatan. Respons mengembalikan struktur array of objects JSON yang memuat sekumpulan daftar fasilitas secara utuh dengan ekstraksi koordinat yang presisi.

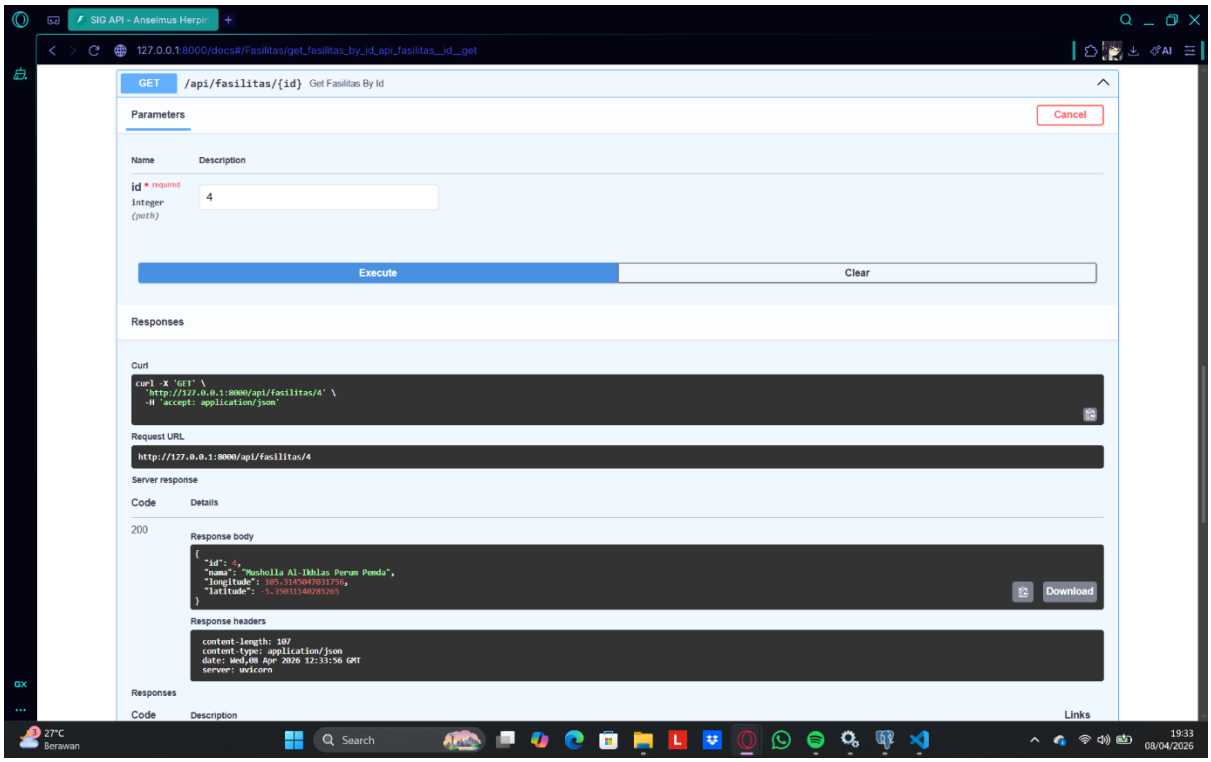
3.2. Endpoint GET /api/fasilitas/{id} (Retrieve Data by ID)

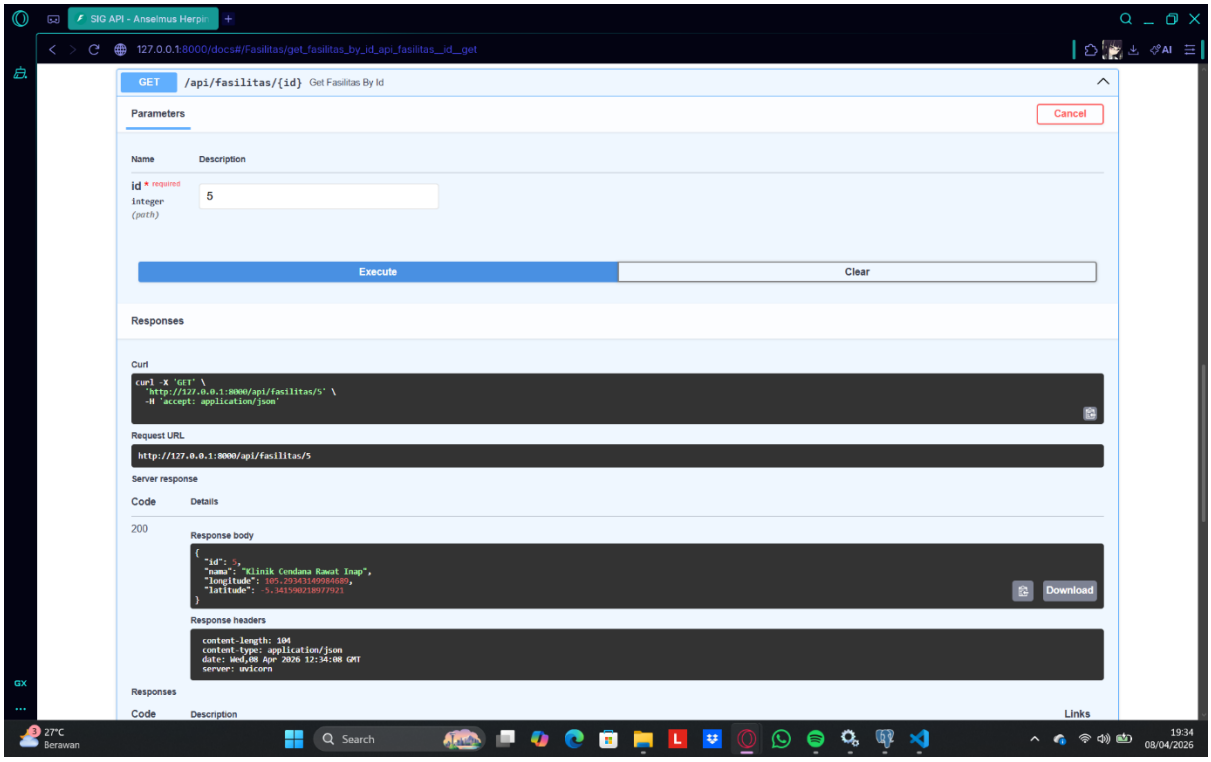
Antarmuka ini didesain untuk mengambil data spesifik berdasarkan pengidentifikasi unik (primary key). Parameter {id} ditangkap dari URL (path parameter) dan diteruskan ke basis data. Guna mencegah kerentanan keamanan berupa serangan SQL Injection, nilai ID tidak dirangkai langsung ke dalam string query, melainkan menggunakan metode parameterisasi ($1) yang secara otomatis dibersihkan (sanitized) oleh driver asyncpg.

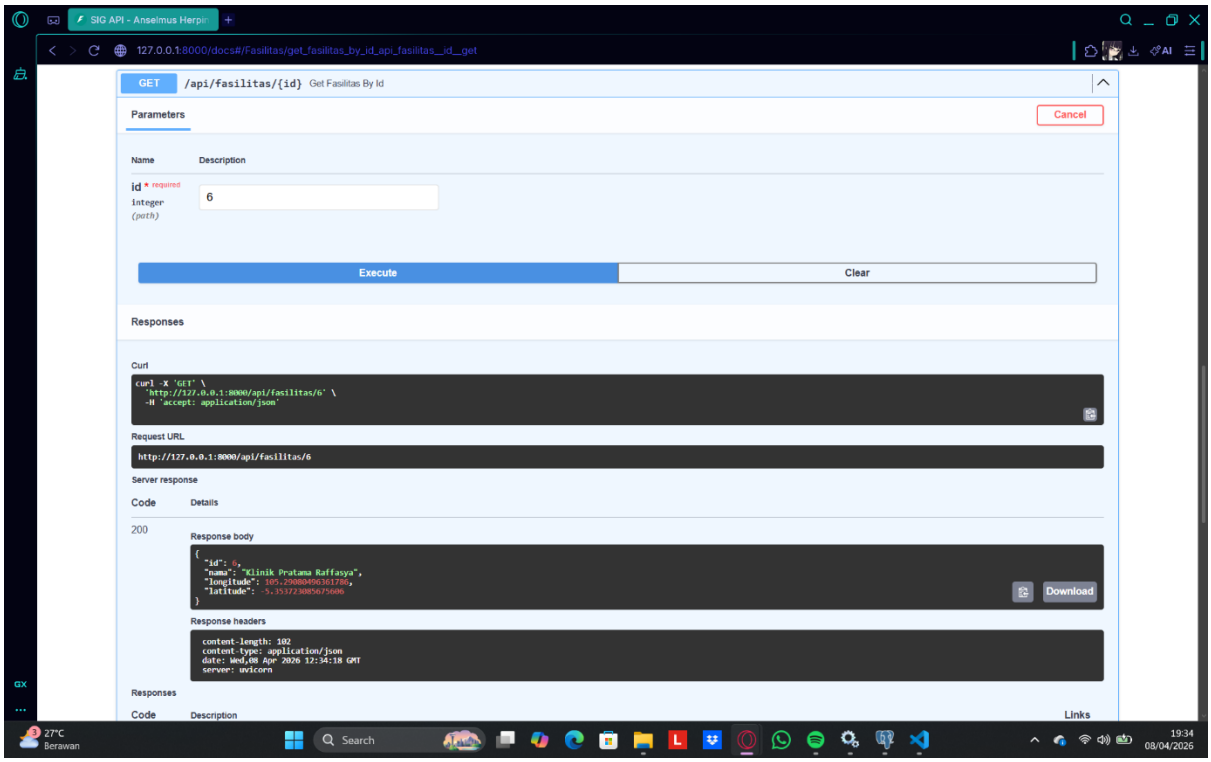





SIG API - Anselmus Herpin
127.0.0.1:8000/docs#/Fasilitas/get_fasilitas_by_id_api_fasilitas__id__get
GET
/api/fasilitas/{id}
Get Fasilitas By Id
Parameters
Cancel
Name
Description
id * required
integer
(path)
3
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/api/fasilitas/3' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/api/fasilitas/3
Server response
Code
Details
200
Response body
{
"id": 3,
"nama": "Klinik Satu Januari",
"longitude": 105.324072373973447,
"latitude": -5.352043580333113
}
Download
Response headers
content-length: 98
content-type: application/json
date: Wed,08 Apr 2026 12:33:42 GMT
server: uvicorn
Responses
Code
Description
Links
27°C
Berawan
Search
19:33
08/04/2026


SIG API - Anselmus Herpin
127.0.0.1:8000/docs#/Fasilitas/get_fasilitas_by_id_api_fasilitas__id__get
GET
/api/fasilitas/{id}
Get Fasilitas By Id
Parameters
Cancel
Name
Description
id * required
integer
(path)
4
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://127.0.0.1:8000/api/fasilitas/4' \
-H 'accept: application/json'
Request URL
http://127.0.0.1:8000/api/fasilitas/4
Server response
Code
Details
200
Response body
{
"id": 4,
"nama": "Musholla Al-Ikhlas Perum Pemda",
"longitude": 105.3145047031756,
"latitude": -5.350311402852265
}
Download
Response headers
content-length: 107
content-type: application/json
date: Wed,08 Apr 2026 12:33:56 GMT
server: uvicorn
Responses
Code
Description
Links
27°C
Berawan
Search
19:33
08/04/2026



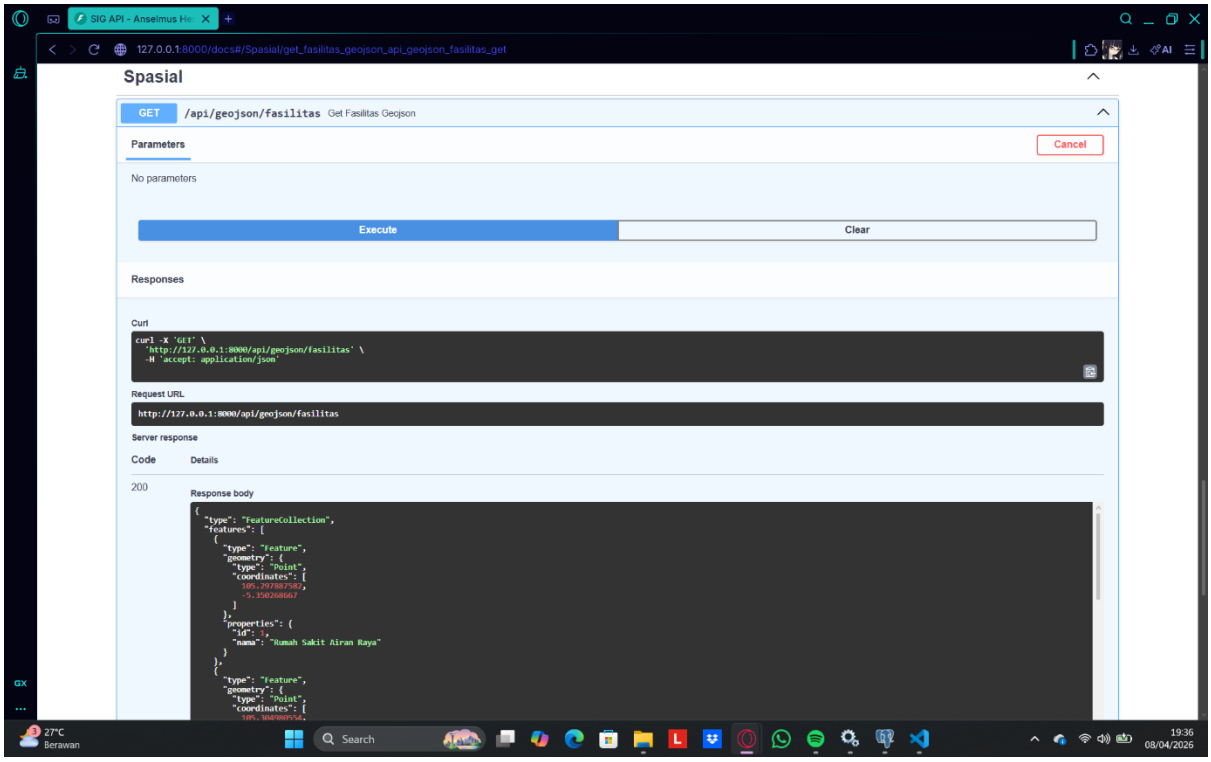


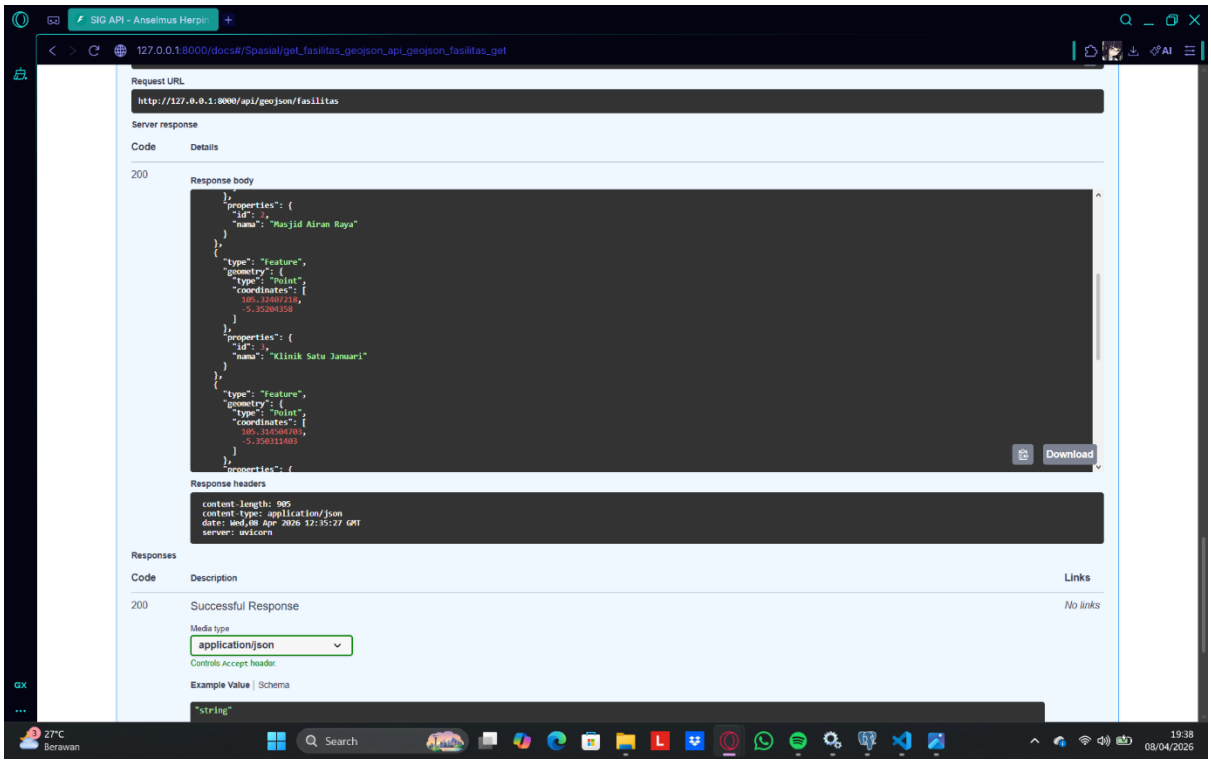
Gambar 3.2. Pengujian Endpoint GET Fasilitas Spesifik berdasarkan ID

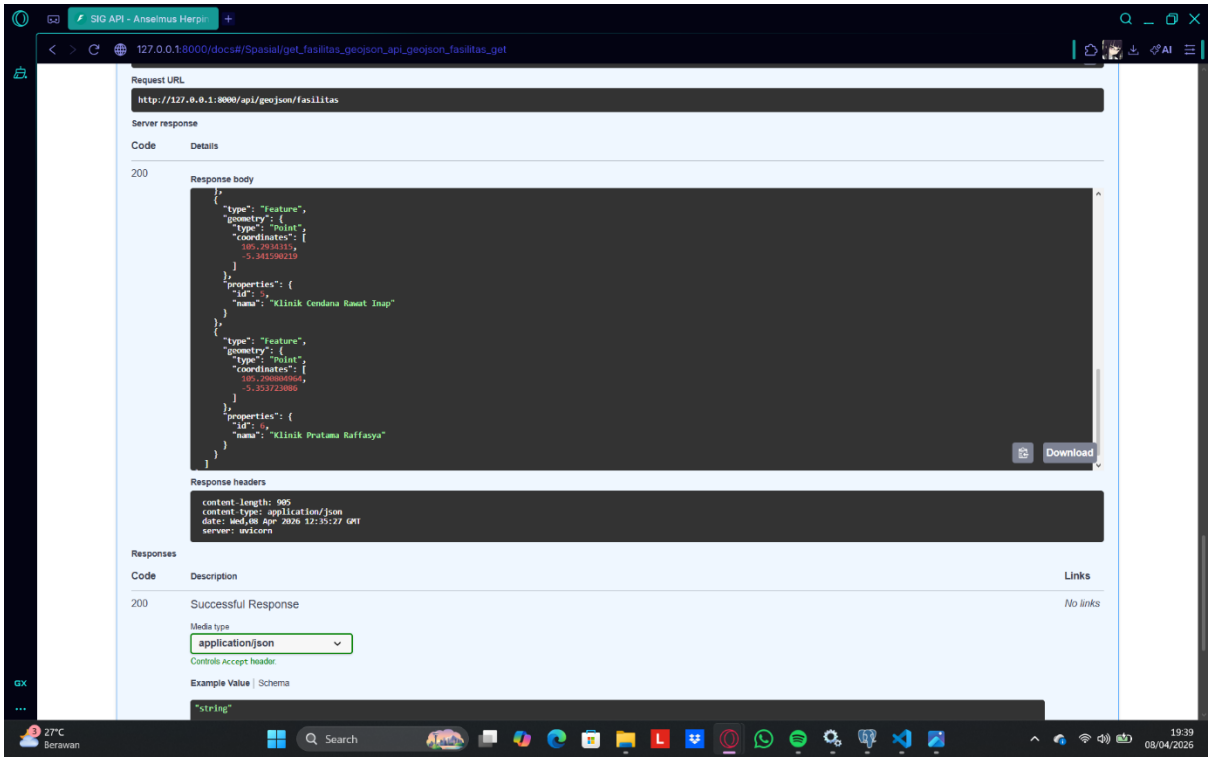
Pengujian menunjukkan sistem berhasil mencocokkan ID yang diminta dengan data di PostGIS dan mengembalikan objek JSON tunggal berisi detail fasilitas terkait dari id:1 hingga id:6 yang telah saya input pada database di tugas sebelumnya.

3.3. Endpoint GET /api/geojson/fasilitas (Generate GeoJSON Format)

Pembuatan endpoint ini merupakan aspek paling fundamental dalam membangun aplikasi berbasis peta web (WebGIS). Kinerja sistem dioptimalkan dengan cara mendelegasikan proses pembentukan format GeoJSON ke dalam mesin basis data (PostGIS), alih-alih mengolahnya di dalam iterasi loop Python. Query kompleks dibangun menggunakan jsonb_build_object untuk menstrukturkan header FeatureCollection, dan dipadukan dengan agregasi jsonb_agg serta fungsi tata ruang ST_AsGeoJSON untuk mengubah data spasial tunggal menjadi objek spasial bersarang.





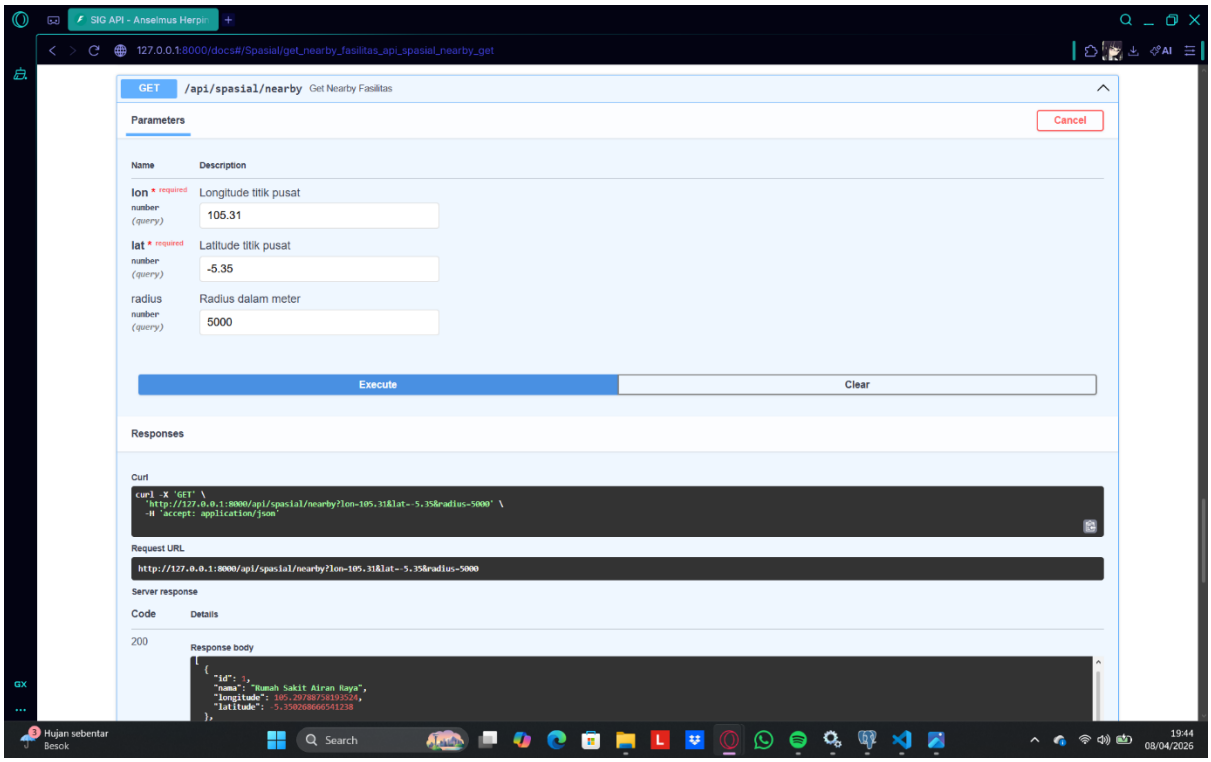


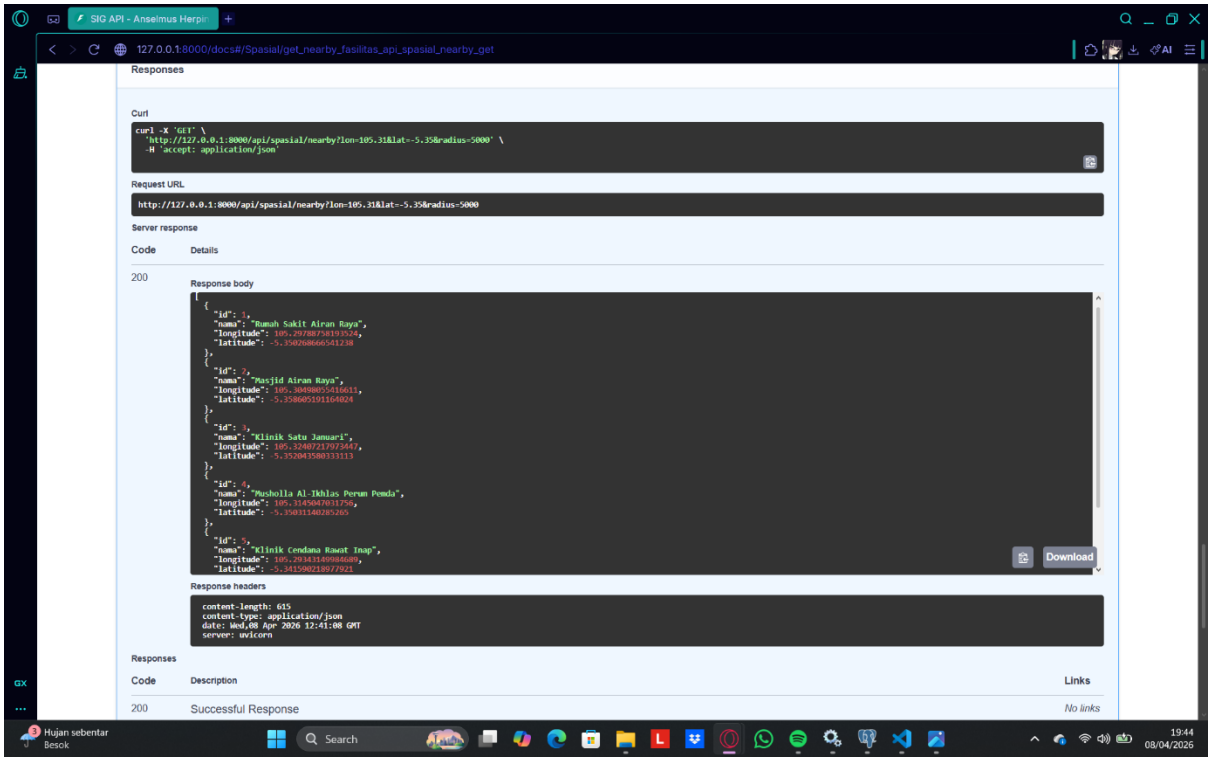
Gambar 3.3. Pengujian Endpoint Output Standar GeoJSON

Hasil luaran (Gambar 3.3) membuktikan bahwa query tersebut sukses dieksekusi dengan Response Code 200. Output yang dihasilkan sangat terstruktur dan secara ketat mematuhi spesifikasi standar RFC 7946 GeoJSON. Data berhasil dipecah menjadi blok geometry yang memuat tipe Point beserta koordinat murninya, serta blok properties yang menyimpan atribut deskriptif seperti nama dan ID. Format matang ini memungkinkan library mapping di sisi frontend (seperti Leaflet atau OpenLayers) untuk langsung merender (plotting) titik-titik tersebut ke atas peta tanpa perlu modifikasi lanjutan.

3.4. Endpoint GET /api/spasial/nearby (Spatial Search & Buffer Analysis)

Endpoint ini mendemonstrasikan kapabilitas API dalam memproses kueri analitik berbasis lokasi (Location Based Services). Sistem menerima tiga parameter input (query parameters): lon (garis bujur pusat), lat (garis lintang pusat), dan radius (jangkauan dalam meter). Inti pemrosesan spasial ini terletak pada pemanfaatan fungsi PostGIS ST_DWithin yang dikombinasikan dengan manipulasi sistem referensi koordinat (SRID 4326). Kolom geometri dilempar (casting) menjadi tipe geography (geom::geography) untuk memastikan perhitungan radius jarak mempertimbangkan kelengkungan bumi (Spheroid) dalam satuan ukur meter yang akurat.



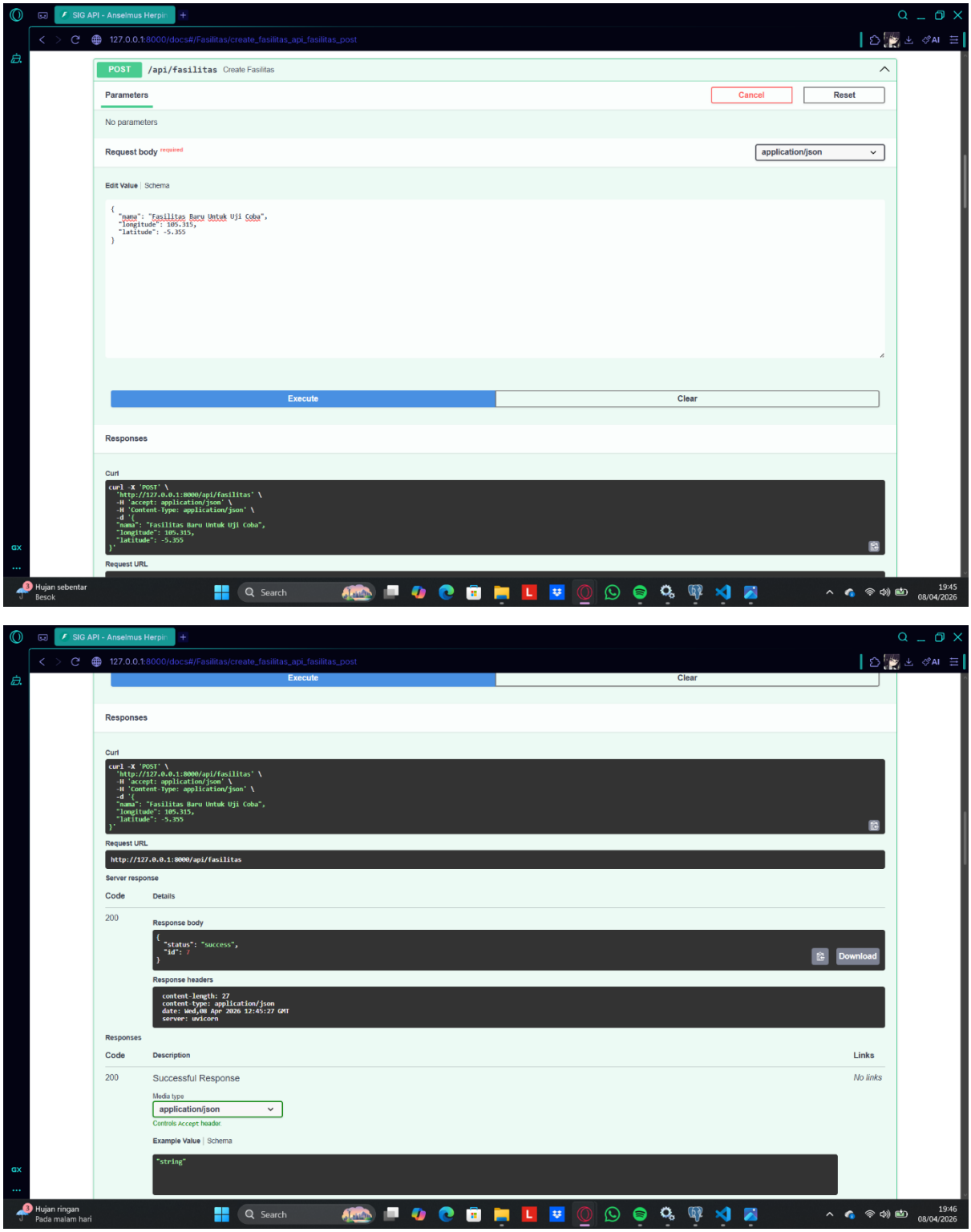


Gambar 3.4. Pengujian Endpoint Analisis Radius Nearby

Pengujian fungsionalitas ini dieksekusi dengan menyematkan titik pusat di kawasan kampus ITERA, Lampung Selatan, dengan radius pencarian sejauh 5000 meter (5 kilometer). Basis data spasial bekerja dengan amat baik dalam menyaring entitas; sistem secara komputasional mengukur jarak dan hanya memunculkan 6 (enam) unit fasilitas yang berpotongan di dalam buffer sirkular virtual tersebut, sementara titik yang berada di luar jangkauan diabaikan.

3.5. Endpoint POST /api/fasilitas (Create New Spatial Data)

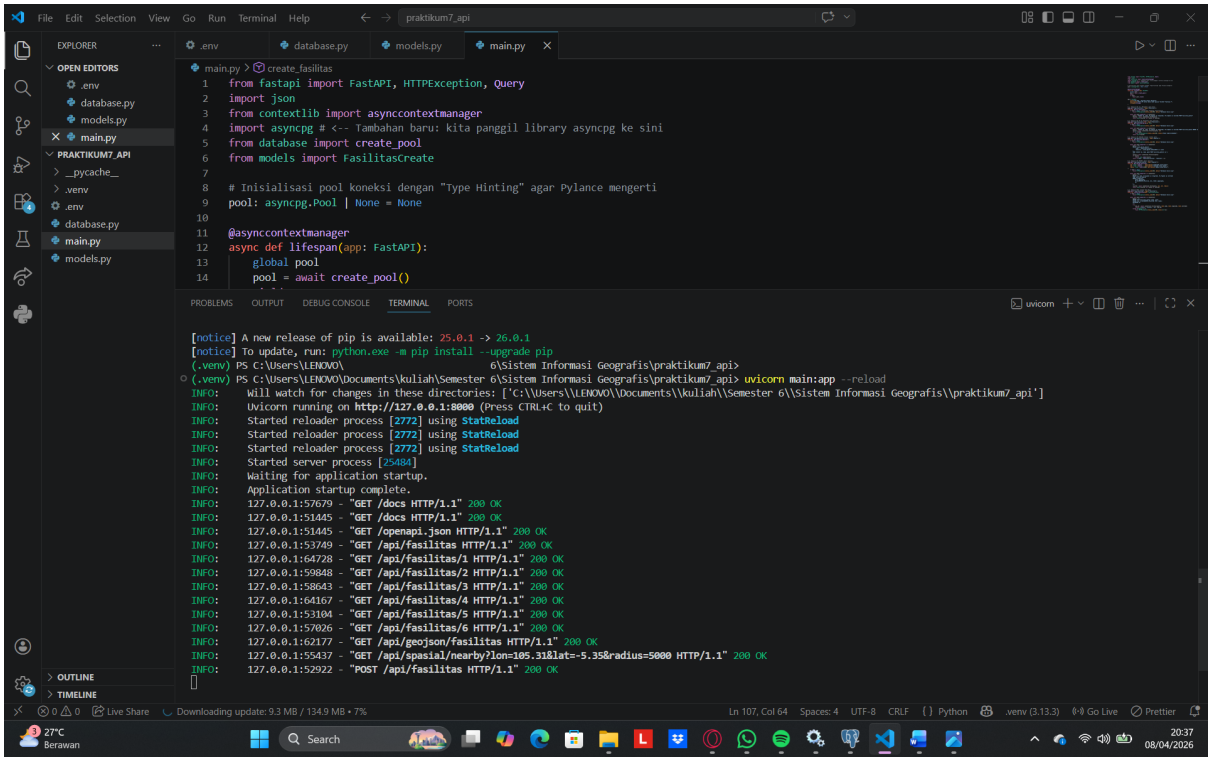
Antarmuka terakhir memfasilitasi injeksi data spasial baru ke dalam tabel PostGIS melalui metode HTTP POST. Sebagai bentuk keamanan dan integritas data, input dari pengguna diwajibkan melewati lapisan validasi skema Pydantic (FasilitasCreate). Hal ini menjamin bahwa tipe data (string dan float) harus terpenuhi sebelum sistem membuka koneksi penulisan ke basis data. Jika data lolos uji, fungsi ST_Point dan ST_SetSRID akan dipanggil secara implisit oleh perintah insert.



Gambar 3.5. Pengujian Endpoint Penyisipan Titik Fasilitas Baru

Berdasarkan request body berisi objek JSON yang disimulasikan sebagai titik uji coba, eksekusi POST mengembalikan respons konfirmasi kesuksesan dengan status 200. Respons tersebut juga menyertakan nomor ID baru yang dihasilkan secara dinamis menggunakan klausa returning id pada SQL. Ini membuktikan bahwa proses konstruksi objek geometri dan commit data ke tabel PostGIS telah terselesaikan secara absolut tanpa terjadi galat translasi data.

Berikut bukti bahwa seluruh rangkaian perintah sukses 200 OK



BAB 4.

KESIMPULAN

Dari keseluruhan proses Latihan dan tugas praktikum ketujuh ini, dapat ditarik kesimpulan bahwa integrasi antara kerangka kerja FastAPI dan basis data PostGIS merupakan solusi arsitektur backend yang sangat tangguh untuk pengembangan Sistem Informasi Geografis (Enterprise GIS). Arsitektur asinkronus yang didukung oleh driver asyncpg memungkinkan komputasi query tata ruang yang berat, seperti konversi GeoJSON dan pencarian proksimitas ST_DWithin dapat dieksekusi dengan kecepatan tinggi dan latensi rendah berkat sifatnya yang non-blocking.

Di samping performa basis data, kualitas API yang dihasilkan juga terjamin oleh validasi ketat dari Pydantic, yang berfungsi sebagai penjaga gerbang (gatekeeper) untuk menolak anomali format data spasial sebelum masuk ke ekosistem PostGIS. Ketersediaan dokumentasi interaktif bawaan juga menyederhanakan siklus pengujian operasional (testing). Secara fungsional, REST API yang dibangun pada praktikum ini telah berhasil mengakomodasi operasi manipulasi data secara utuh, menghasilkan representasi spasial berstandar industri (GeoJSON), dan siap untuk diintegrasikan secara seamless sebagai tulang punggung suplai data pada perancangan antarmuka WebGIS interaktif di tahap selanjutnya.

Link Repository GitHub:
https://github.com/forkaton/Praktikum7_SIG_123140020_Anselmus-Herpin-Hasugian.git

Link Gdrive ZIP:
https://drive.google.com/drive/folders/1QEjRWcXiWJjolOwhajIEPEATL-FhT0IE?usp=sharing